Міністерство освіти та науки України
Національний технічний університет України «Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ
з дисципліни «Програмування веб-застосувань»
лабораторна робота №10
«Клієнтське програмування з бібліотеками та фреймворками (Vue/React)»

Виконав:
Студент II курсу
групи ІП-52
Демура Є.Р.
Перевірив:
Ковтунець О.В.

2026

Мета роботи: Обрати один із двох популярних клієнтських фреймворків - Vue 3 чи React 18 - і підключити

його через CDN, без збірки й Node.js. Перенести список елементів вашого варіанта, що дотепер

рендерився вручну через DOM API (практикум 7), на компонентний підхід: окремий компонент

отримує дані через props і рендерить їх декларативно (v-for у Vue чи map() у React), а одну

інтерактивну дію реалізувати через подію фреймворку, яка автоматично оновлює реактивний

стан і перемальовує інтерфейс - без жодного ручного querySelector чи innerHTML для цієї

ділянки.

Варіант 7

Завдання:

7. Список бажань (wishlist)	Компонент WishItem, props: name, price, priority. Подія: клік по чекбоксу «придбано» передає toggle(id) батьківському компоненту, який оновлює масив бажань.

Хід роботи

Крок 1. Обрати фреймворк. Vue чи React - обґрунтуйте вибір одним реченням у коміті чи
коментарі на початку файлу.

Обрав я Vue 3, бо його шаблони пишуться прямо в HTML і не потребують Babel для JSX, а вбудована реактивність дозволяє змінювати масив бажань напряму, що для невеликого застосунку без збірки є найпростішим рішенням.

Крок 2. Підключити CDN-скрипти. Додайте потрібні <script>-теги в HTML (для React - три
теги: react, react-dom, babel standalone).

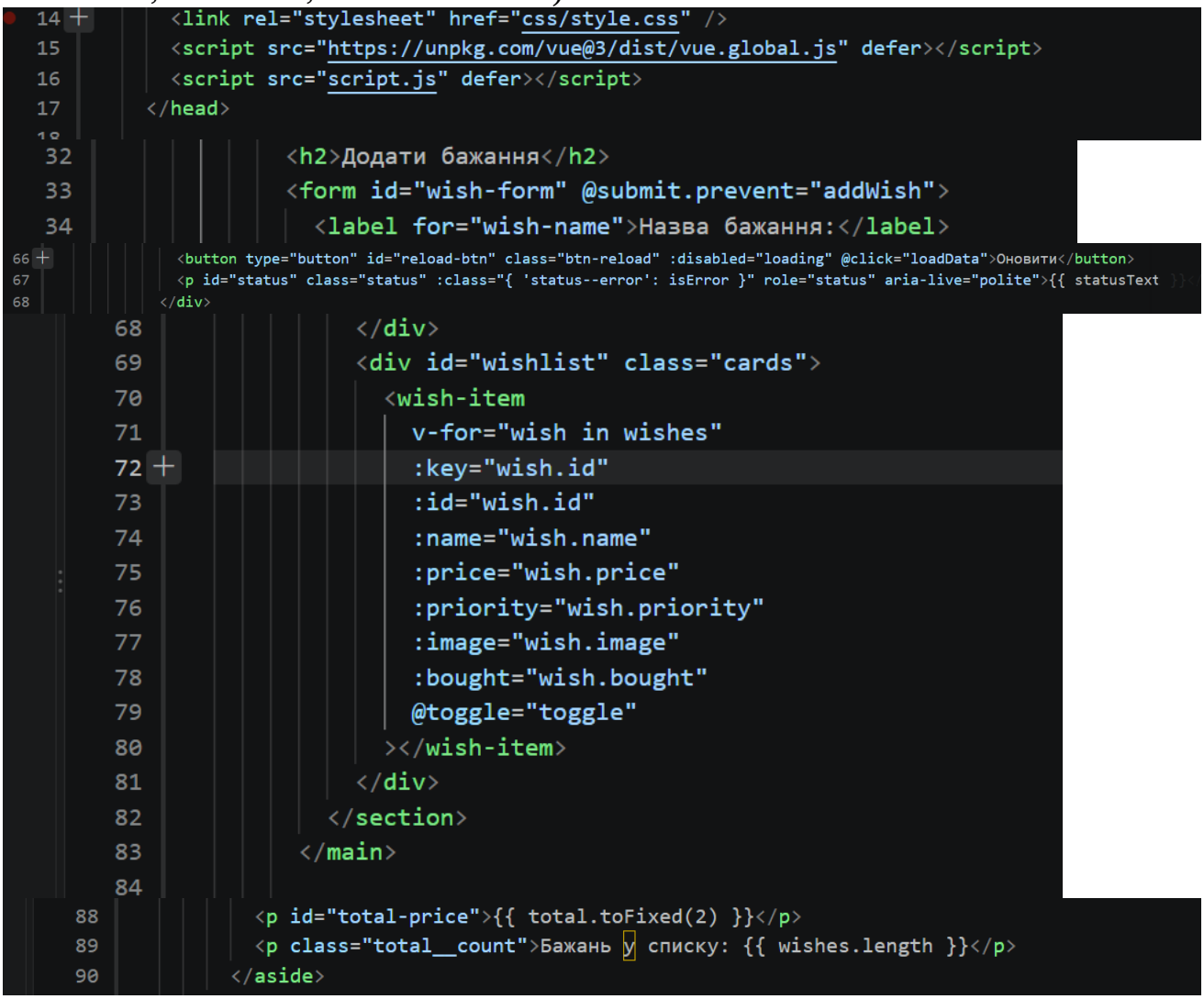


Рис 2.1, 2.2, 2.3, 2.4, 2.5, CDN-скрипти та script теги

Крок 3. Перенести дані в реактивний стан. Масив об'єктів вашого варіанта (практикум 6, чи

результат fetch із практикуму 9) помістіть у data() (Vue) чи useState (React) - не в звичайну
зовнішню змінну.

```
data() {
  return {
    wishes: [
      {
        id: 1,
        name: "Новий ноутбук",
        price: 109999,
        priority: "високий",
        bought: false,
        image:
          "https://content.rozetka.com.ua/goods/images/big/677460228.jpg",
      },
      {
        id: 2,
        name: "Навушники",
        price: 3399,
        priority: "середній",
        bought: false,
        image:
          "https://content.rozetka.com.ua/goods/images/big/595395804.jpg",
      },
      {
        id: 3,
        name: "Книга з програмування",
        price: 1500,
        priority: "низький",
        bought: false,
        image:
          "https://work.njae.me.uk/content/images/size/w2000/2018/04/grokking-algorithms.png",
      },
    ],
    newName: "",
    newPrice: null,
    newPriority: "середній",
    loading: false,
    statusText: "",
    isError: false,
  };
},
```

Рис. 3. Масив об’єктів wishes лежить у data()

Крок 4. Створити компонент елемента/списку. Опишіть окремий компонент за вашим
варіантом, що приймає дані через props і не звертається до глобальних змінних напряму.

```
const WishItem = {
  props: {
    id: { type: Number, required: true },
    name: { type: String, required: true },
    price: { type: Number, default: null },
    priority: { type: String, default: "середній" },
    image: { type: String, default: "" },
    bought: { type: Boolean, default: false },
  },
  emits: ["toggle"],
```

Рис. 4. WihsItem приймає props

Крок 5. Реалізувати рендер циклом. v-for (Vue) чи map() у JSX (React) - з обов'язковим
:key/key за унікальним id елемента.

```
<wish-item
  v-for="wish in wishes"
  :key="wish.id"
  :id="wish.id"
  :name="wish.name"
  :price="wish.price"
  :priority="wish.priority"
  :image="wish.image"
  :bought="wish.bought"
  @toggle="toggle"
></wish-item>
```

Рис. 5. Рендер з циклом

Крок 6. Реалізувати інтерактивну подію варіанта. Обробник у дочірньому компоненті
викликає $emit (Vue) чи callback-prop (React); батьківський компонент оновлює стан новим
значенням (не мутацією напряму в React).

```
      :data-priority="priority"
    >
      <img v-if="image" :src="image" :alt="name" />
      <h3 class="wish__name">{{ name }}</h3>
      <p v-if="price !== null" class="wish__price">{{ price.toFixed(2) }} грн</p>
      <p class="wish__priority-label">Пріоритет: {{ priority }}</p>
      <label class="wish__bought">
        <input type="checkbox" :checked="bought" @change="$emit('toggle', id)" />
        Придбано
      </label>
    </article>
  `,
};
```

Рис. 6. Подія варіанта(Vue)

Крок 7. Прибрати ручний DOM-код. Видаліть чи явно закоментуйте функцію рендеру з
практикуму 7 для цієї ділянки інтерфейсу - з поясненням, що її замінено компонентом.

```
/*
  Функцію ручного рендеру з практикуму 7 замінено компонентом WishItem і директивою v-for.
  Vue сам створює й оновлює картки, тому querySelector, createElement та innerHTML
  для списку більше не потрібні.

function renderWishes(items) {
  listContainer.innerHTML = '';
  items.forEach((wish, index) => {
    const card = document.createElement('article');
    ...
    listContainer.append(card);
  });
}
*/
```

Рис. 7. Закоментована функція

Крок 8. Перевірити автоматичне оновлення. Змініть стан (клік по інтерактивному елементу) і
переконайтеся, що список перемальовується сам, без жодного ручного querySelector/innerHTML.

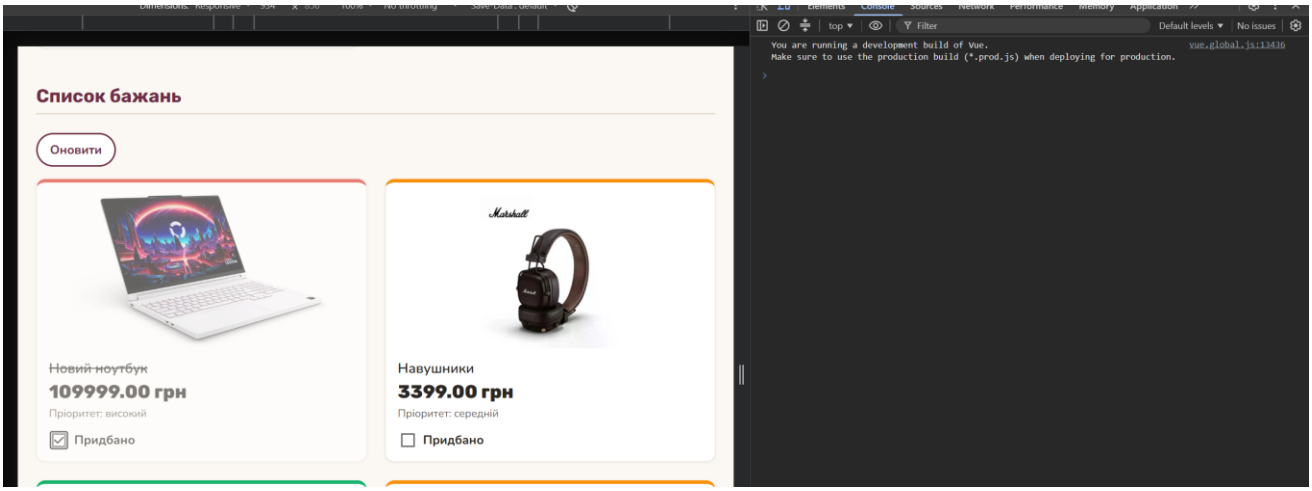


Рис. 8. Зміна стану без помилок

Крок 9. Перевірити у DevTools. Відкрийте Vue DevTools чи React DevTools і переконайтеся,
що дерево компонентів і поточний стан відповідають очікуваним.

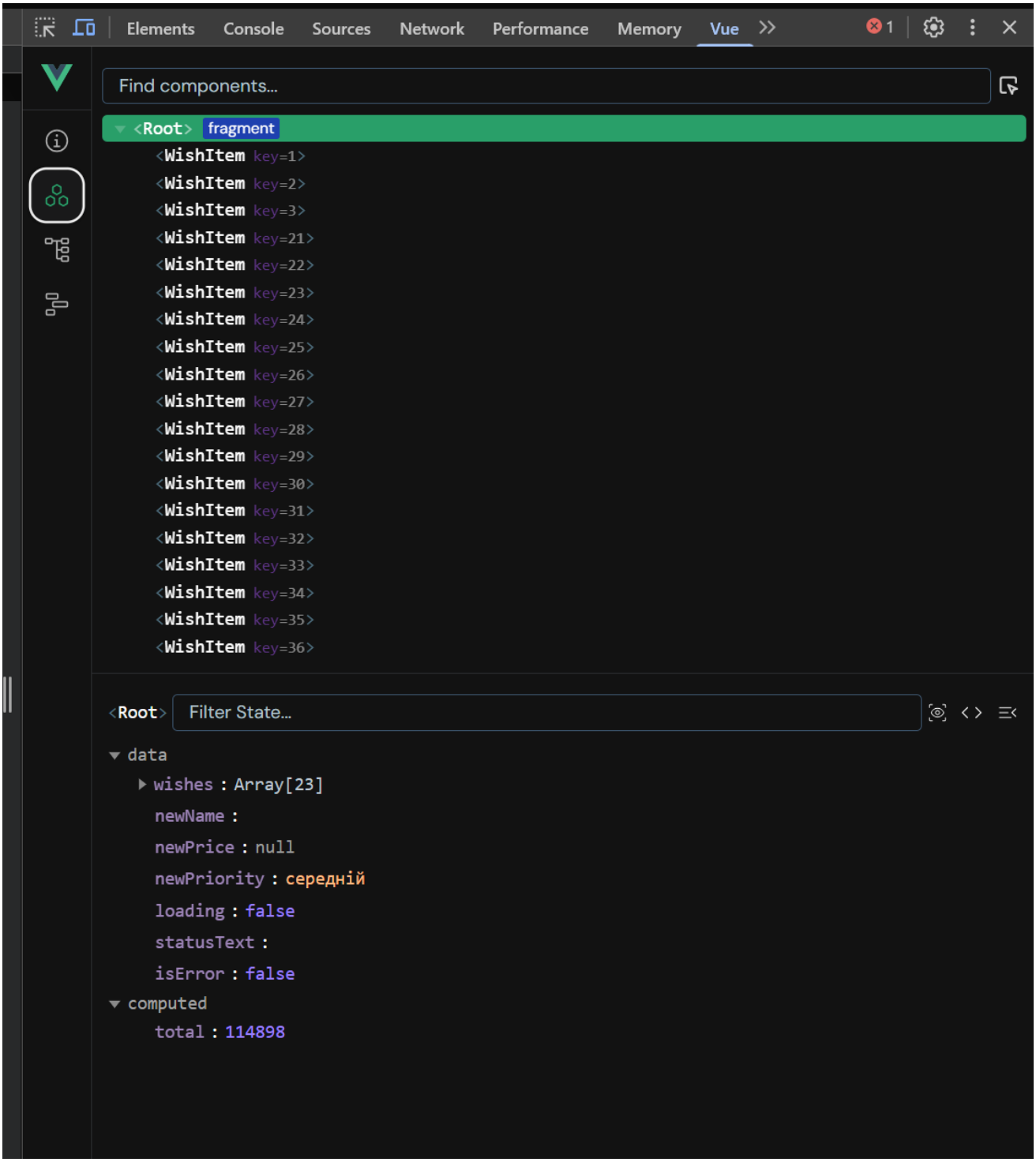


Рис. 9. Поточний стан відповідає очікуваному

Крок 10. Прокоментувати й зберегти. Додайте короткий коментар про обраний фреймворк і
причину вибору, зробіть коміт із описовим повідомленням.

ПОСИЛАННЯ НА GITHUB: Посилання

Висновок: У результаті виконання практичної роботи список бажань переведено з ручного рендеру через DOM API на компонентний підхід із фреймворком Vue 3, підключеним через CDN без збірки. Дані зберігаються в реактивному стані `data()`, а кожне бажання відображає окремий компонент `WishItem`, що отримує дані лише через props і рендериться директивою `v-for` з унікальним `:key`. Клік по чекбоксу «Придбано» передає подію `toggle(id)` батьківському компоненту, який оновлює масив, після чого Vue автоматично перемальовує інтерфейс. Підсумкова сума реалізована як обчислюване значення `computed` і завжди відповідає поточним даним. Ручний DOM-код (`querySelector`, `createElement`, `innerHTML`) повністю замінено декларативними шаблонами. Дерево компонентів і стан перевірено у Vue DevTools.